

GamutX

Variables

Python Basics

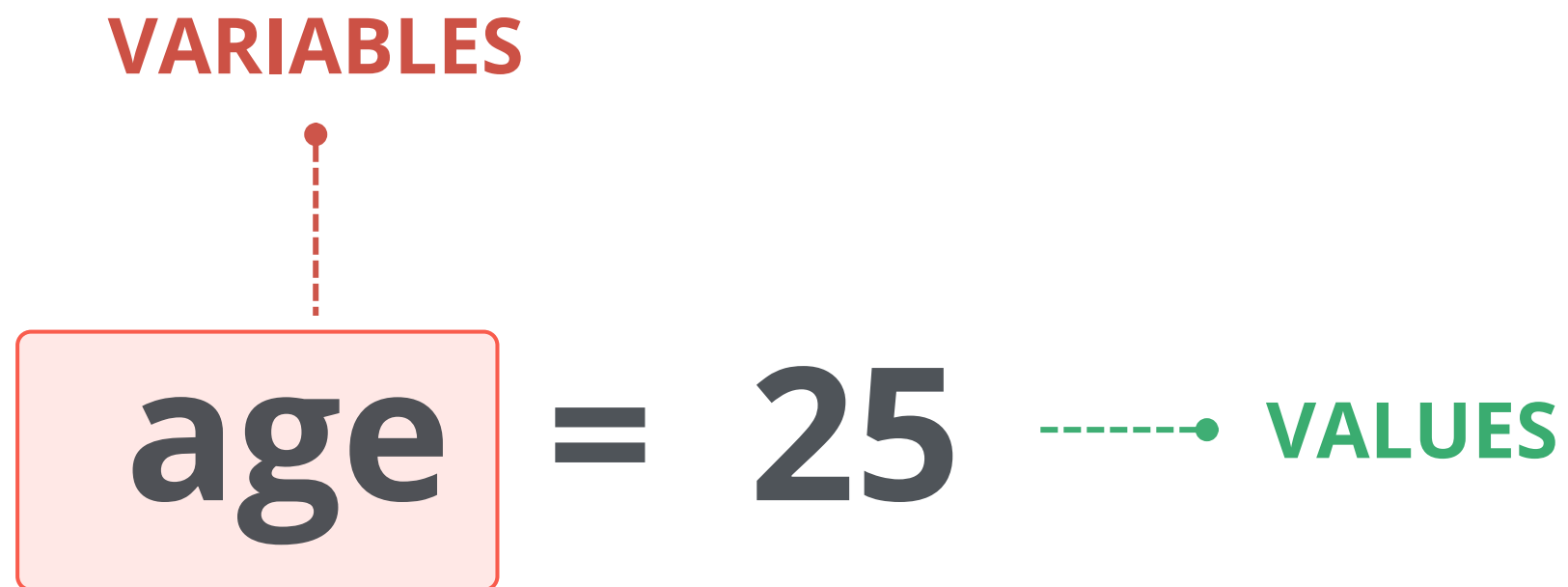


Understanding Variables

- What is a Variable?
- Creating Variables
- What is an Identifier?
- What Are Data Types?
- Common Data Types in Python
- What are Literals

What is a Variable?

- A variable is a **container that stores data values** in a program.
- A variable is just a **name given to a memory location** in your program.

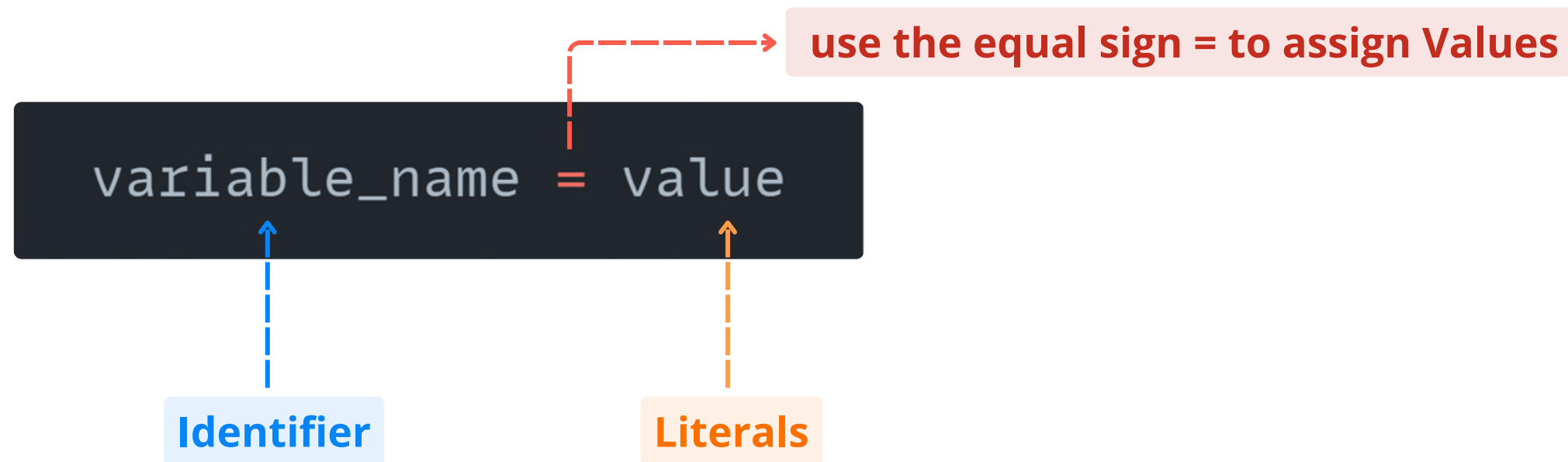


- It's used to **store, retrieve, and change values** when needed.

Creating Variables

- We use the **equal sign =** to assign a value to a variable.

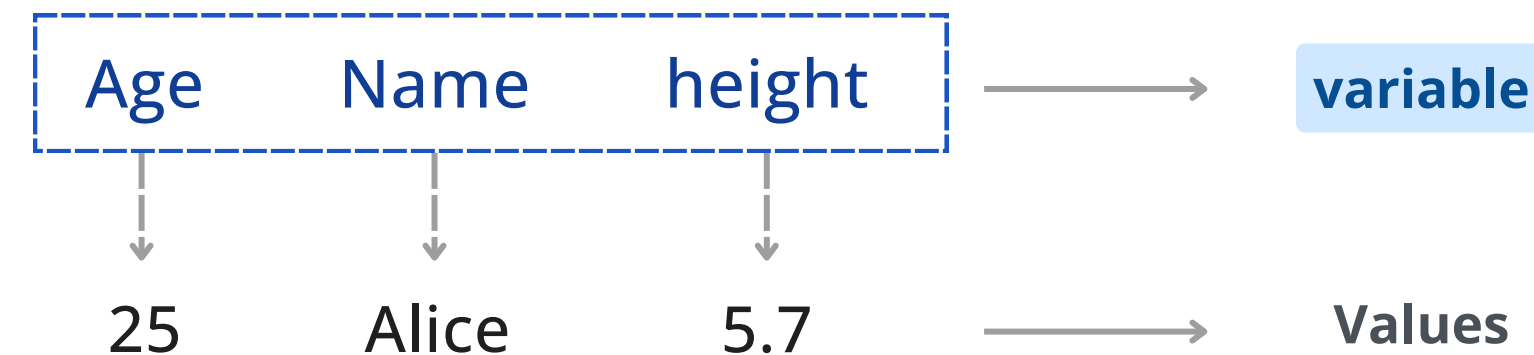
Syntax →



Example

```
age = 25
name = "Alice"
height = 5.7
```

Here



We can now use these variables anywhere in your program — **print** , **update** , or **perform operations** on them.

What is an Identifier?

- In simple terms, an **identifier is just the name we use in our code** to label something.
- That “something” could be:
 - **A variable**
 - **A function**
 - **A class**
 - **Or other objects in Python**

Let's look at an example:

```
age = 25  
name = "Alice"  
height = 5.7
```

In this example:

- age
- name
- height

These are all identifiers.

They are the names we chose to give to certain pieces of data.

Using identifiers:

- Keeps your code **shorter**
 - Makes your code **easier to update**
 - Helps you **avoid repeating** values
 - Writing `user_name = "Alice"` is more meaningful than just `"Alice"` on its own
- 

What Are Data Types?

- When writing code, we work with different types of information:
 - Numbers -----> 10, 3.14)
 - Text -----> Shot_01", "Render complete"
 - Boolean values -----> is the file loaded?
- To handle this properly, Python needs to know **Data Types**
- A **Data Type** is a label Python uses to understand:
 - What kind of value it is
 - What operations can be done with it (e.g., add, compare, slice)

Real-World Comparison: A Library

- Let's compare this with a Library and Python terms



In a library

- **Books** are arranged into different sections:
 - Fiction,
 - Non-Fiction,
 - Biographies, etc.
- This helps people find and use the right kind of book.

Python Terms

- Python arranges **Data into Types**:
 - **Integers**
 - **Strings**
 - **Booleans, etc.**
- It helps program know how to handle each piece of data.

Common Data Types in Python

Here are the four most common types you'll use in Python:

a. Integer (int)

- Whole numbers
- No decimal point
- **Example:** 10, -3, 250

c. String (str)

- Text or sequence of characters
- Enclosed in quotes: "Hello", 'Shot_001'
- **Example:** "Gamma TD", "123"
- **Note:**
 - even though this looks like a number,
 - it's still a string if in quotes

b. Float (float)

- Numbers with a decimal point
- Used when you need more precision
- **Example:** 3.14, 0.5, 100.0

d. Boolean (bool)

- Only two possible values: True or False
- Used for conditions and decisions
- **Example:** Is a shot approved? → True or False

Purpose of Data Types

Both in real life and in programming.

◆ In a Library

- Categories help you locate the right kind of book quickly.
- For example:
 - Want a science fiction novel? → Go to the Science Fiction section.
 - Looking for biographies? → Head to the Biographies section.
- This avoids confusion and saves time.

◆ In Programming

- Data types serve the same purpose — they help you work with the right kind of data.
- Examples:
 - Doing math like addition or division? → Use int or float.
 - Storing names, messages, or file paths? → Use str.
 - Need a true/false answer, like “is the file found?” → Use bool.

What are Literals

- In programming, a **literal** is a fixed value written directly into your code.
- It's not calculated or stored in a variable — it is the actual value.
- Just like in real life, we often use clear, specific values:
 - “I have 10 apples” → 10 is a literal.
 - “The meeting is at 5 PM” → 5 PM is a literal expression of time.

Types of Literals

Let's see main types of literals you'll use.

- Integer Literals
- Float Literals
- String Literals
- Boolean Literals

• Integer Literals

- These are whole numbers (no decimals).
- Used for things you count:
 - frames, objects, shots, etc.
- Examples : 45, -8, 1000

Real-life example:

- You count 15 chairs in a room.
- Number — 15 — is an integer literal.

• String Literals

- These are sequences of characters,
- surrounded by quotes.
- Can include letters, numbers, or symbols.
- Examples : "Hello", 'Shot_05', "123"

Real-life example:

- A greeting card says: **"Happy Birthday"**

└───→ string literal.

• Float Literals

- These are numbers with decimal points.
- Used when precision is important:
 - measurements, percentages, etc.
- Examples : 3.14, 0.99, 2.5

Real-life example:

- A recipe calls for 2.5 cups of flour → 2.5 is a float literal.

• Boolean Literals

- These represent truth values:
- only two possible values — True and False.
- Commonly used for conditions:
 - yes/no, on/off, success/failure.
- Examples : True, False

Real-life example:

- A light is either on (True) or off (False).

Literals Summary

Literal Type	Example (Code)	Used For	Real-World Parallel
Integer	25	Counts, IDs	25 chairs
Float	3.5	Precise values, measurements	3.5 hours of rendering
String	"Scene_01"	Names, text, descriptions	Writing "Happy Birthday" on a card
Boolean	True	Logical conditions	Light switch is ON (True)

Key Points to Remember

Variable

Container that stores Data Value

name = "SpaceX"

↑
↓

→ string literal.

Identifiers

names used to label **variables**, **functions**, **classes**, or other elements in code.

Literals

Literals are **fixed values** written directly in the code

Data Types

Integer

3, 4, 10

Float

2.76, 3.14

String

"earth", "A"

Boolean

True, False

Concept Practice Task

Storing Personal Information

```
name = "John Doe"  
age = 30  
height = 5.9
```

```
print(name)  
print(age)  
print(height)
```

```
#Output:  
John Doe  
30  
5.9
```

Storing Product Details

```
product_name = "Laptop"  
price = 999.99  
stock = 25
```

```
print(product_name)  
print(price)  
print(stock)
```

```
#Output:  
Laptop  
999.99  
25
```

Storing Marks of a Student

```
math_marks = 85  
science_marks = 90  
english_marks = 78
```

```
print(math_marks)  
print(science_marks)  
print(english_marks)
```

```
#Output:  
85  
90  
78
```

Storing Temperature and Weather Information

```
temperature = 28.5  
weather_condition = "Sunny"
```

```
print(temperature)  
print(weather_condition)
```

```
#Output:  
28.5  
Sunny
```

Common Built-in Functions in Python

- `print()` Function
- `input()` Function
- `len()` Function
- `type()` Function
- Arithmetic Operations with Variables

print() Function

- To **display the value stored in a variable**, we use the **print() function**.
- This function **outputs the value to the screen**.

Printing Variables

```
# Defining variables
name = "Alice"
age = 25

# Printing variables
print(name) # Alice
print(age)  # 25
```

- We can do the same thing with other types of data, like strings (text) or floating-point numbers (decimal numbers).

Concept Practice Task

GamutX

Printing Variables

```
# Defining variables
name = "Alice"
age = 25

# Printing variables
print(name) # Alice
print(age)  # 25
```

Printing Boolean Values

```
is_sunny = True
is_raining = False

print(is_sunny)    # True
print(is_raining)  # False
```

Boolean

Printing Numbers

```
# Printing integer and float numbers

print(25)          # 25
print(3.14159)     # 3.14159
```

Combine Example

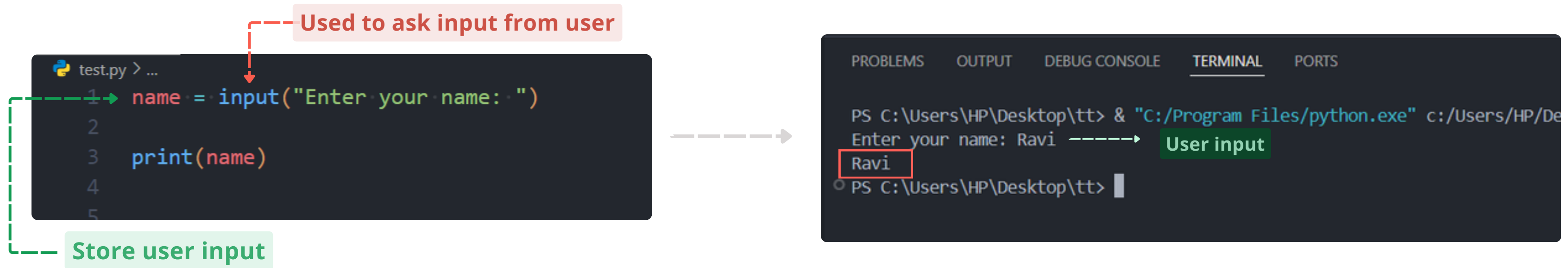
```
age = 25 -----> Int
print(age)  # Output: 25

name = "Alice" -----> String
print(name) # Output: Alice

height = 5.7 -----> Float
print(height) # Output: 5.7
```

input() Function

- The **input()** function is **used** when we **want to ask the user to type something**.
- It makes the **program pause** and **wait until the user types** and presses Enter.



How It Works

Prompt the User

- We can add a message inside `input()`
- To tell the user what to type.

Input is Always a String

- Whatever the user types — text or numbers
- Python treats it as a string

Storing the Input

- Whatever the user types gets saved into a variable
- We can use it later in your program.

Concept Practice Task

Asking for the User's Name

```
name = input("Enter your name: ")
print("Hello!")
print(name)
```

Output:

```
# Enter your name: Alice
# Hello!
# Alice
```

Asking for the User's Age

```
age = input("How old are you? ")

print("You are")
print(age)
print("years old.")
```

Output:

```
# How old are you? 25
# You are
# 25
# years old.
```

Asking Two Numbers and Showing Them

```
num1 = input("Enter the first number: ")
num2 = input("Enter the second number: ")

print("You entered:")
print(num1)
print("and")
print(num2)
```



Output

```
# Output:
# Enter the first number: 5
# Enter the second number: 10
# You entered:
# 5
# and
# 10
```

len() Function

- The **len()** function is used to **count** how many characters are in a string.

```
x = "hello"
```

```
print(len(x))
```

```
# Output: 5
```

Why is the output 5?

H E L L O

1

2

3

4

5

←-----

Count

Len() Function counts

- Letters -----> a, b, c, d, e, f | A, B, C, D, E
- Numbers -----> 1, 2, 3, 4, 5, 6, 7, 8, 9, 0
- Spaces -----> hello world
- Symbols -----> @, %, \$, (, [, !

Concept Practice Task

len() with an Empty String

```
empty_string = ""  
  
length_of_empty_string = len(empty_string)  
  
print(length_of_empty_string) #Output - 0
```

len() with a String Variable

```
name = "Alice"  
  
length_of_name = len(name)  
  
print(length_of_name) #Output: 5
```

len() with a String Containing Space

```
word_with_spaces = "Hello World"  
  
length_of_word_with_spaces = len(word_with_spaces)  
  
print(length_of_word_with_spaces) #Output: 11
```

len() with a Sentence (String)

```
sentence = "Python programming is fun."  
  
length_of_sentence = len(sentence)  
  
print(length_of_sentence) #Output: 26
```

type() Function

- The type() function is used to **check what kind of data a variable is holding**.
- In Python, **data can be different types**:
 - "hello" ·---→ **str (string)**
 - 42 ·---→ **int (integer)**
 - 3.14 ·---→ **float (decimal)**
 - True/ False ·---→ **bool (boolean)**

Example

```
x = "hello" -----> String Data Type  
  
print(type(x))    # Output: <class 'str'>
```

- The **variable x holds** the text **"hello"**.
- **"Hello" → String**

Concept Practice Task

GamutX

type() with a String

```
name = "Alice" -----> String
```

```
name_type = type(name)
```

```
print(name_type) #Output: <class 'str'>
```

type() with an Integer

```
age = 25 -----> Int
```

```
age_type = type(age)
```

```
print(age_type) #Output: <class 'int'>
```

type() with a Float

```
height = 5.7 -----> Float
```

```
height_type = type(height)
```

```
print(height_type) #Output: <class 'float'>
```

type() with a Boolean

```
is_student = True -----> Boolean
```

```
is_student_type = type(is_student)
```

```
print(is_student_type) #Output: <class 'bool'>
```

Arithmetic Operations with Variable

Just like in math class, you can use variables to perform basic arithmetic operations like

Addition (+)

```
a = 10
b = 5

result = a + b

print(result) #Out: 15
```

Subtraction (-)

```
a = 20
b = 8

result = a - b

print(result) #Out: 12
```

Multiplication (*)

```
a = 4
b = 7

result = a * b

print(result) #Out: 28
```

Division (/)

```
a = 20
b = 4

result = a / b

print(result) #Out: 5.0
```

Concept Practice Task

Combining Operations

```
a = 10
```

```
b = 5
```

```
addition = a + b
```

```
subtraction = a - b
```

```
multiplication = a * b
```

```
division = a / b
```

```
print("Addition:", addition)
```

```
# Output: Addition: 15
```

```
print("Subtraction:", subtraction)
```

```
# Output: Subtraction: 5
```

```
print("Multiplication:", multiplication)
```

```
# Output: Multiplication: 50
```

```
print("Division:", division)
```

```
# Output: Division: 2.0
```

Naming Variables

- Importance of Naming Variables
- Naming Conventions Types
- Guidelines for Naming Variables
- Common Mistakes
- Best Practices for Naming Variables

Importance of Naming Variables

- **Good variable names** help you — and others — **understand** what your **code is doing, even after a long time.**
- It makes your code:
 - **Easier to read**
 - **Easier to debug**
 - **Easier to maintain**

Run Entire Script

- Using descriptive names helps **explain** what the variable is storing. great for team projects

✗ Bad Practice

```
x = 25  
y = 15
```

"What are x and y for?"

✓ Good Practice

```
age = 25  
experience_years = 15
```

"Here we know what **each number** means!"

Avoiding Confusion

- Vague names like `data` or `info` don't explain the variable's purpose, leading to mistakes and harder debugging.

✗ Bad Practice

```
data = "John"  
info = "Doe"
```

What is `data`? What does `info` mean here?

✓ Good Practice

```
first_name = "John"  
last_name = "Doe"
```

“ Clear intent, no guessing needed.”

Ease of Maintenance

- Meaningful variable names make it easier to understand, update, or debug our code—even months later.

✗ Bad Practice

```
x = 500  
y = x * 1.2
```

Later, we might forget what `x` and `y` were used for.

✓ Good Practice

```
base_salary = 500  
bonus = base_salary * 1.2
```

it's easy to understand the calculation and make changes if needed.

Consistent Naming Standards

- Using a **consistent naming** style like **snake_case** makes your code
 - **predictable**,
 - **professional**, and
 - **easier** for teams to read and manage.

✗ Bad Practice

```
x = 25  
y = 15
```

Mixing styles (snake_case, camelCase) makes **code harder to follow**.

✓ Good Practice

```
age = 25  
experience_years = 15
```

This is **clean**, **consistent**, and **easier to read**.

Naming Conventions Types

- **Variables name** with **more than one word** can be difficult to read.
- There are ways you can use to **make them readable**, using **Naming Conventions**.

Types

- There are **three common Types** you'll come across

camelCase

- The **first word** is **lowercase**.
- Every **next word starts** with a **capital letter**.
- **No spaces or underscores**.

```
camelCaseExample = "This is camelCase"
```

- Common in **JavaScript**

PascalCase

- **Every word starts with a capital letter**, including the first one.
- **No spaces or underscores**.

```
PascalCaseExample = "This is PascalCase"
```

- Often used for **Class Names**

snake_case

- All letters are lowercase.
- Words are separated by underscores (_).

```
snake_case_example = "This is snake_case"
```

- This is the preferred style in Python, especially for variable and function names.

✓ Which One Should You Use?

- Use “snake_case” for **variable** and **function names**.
- Use “PascalCase” for **class names**.

Guidelines for Naming Variables

Rule 1:

- The **first character** must be a **letter (A-Z or a-z)** or an **underscore (_)**

✓ Valid

```
my_variable = 10
_age = 25
```

```
print(my_variable) # Output: 10
print(_age)        # Output: 25
```

✗ Invalid

```
1variable = 20
```

We cannot start a variable name with a number.

Rule 2:

- We can **only use letters, numbers, and underscores (_)** in the names

✓ Valid

```
var_1 = 100
hello_123 = "Hi"
```

✗ Invalid

```
my-var = 50
```

No other characters and Symbols are allowed

Rule 3 :

- **No white space or special character is allowed** to be used **inside variable name**.

✓ Valid

```
user_age = 30  
myVariable = "John"
```

✗ Invalid

```
my variable = 45  
hello@world = 90
```

contains '@'

contains a space

Rule 4 :

- **Identifiers names are case sensitive**, for example, **Fruits and fruits is not same**.

✓ Valid

```
Fruits = "Apple"  
fruits = "Banana"
```

Both Different

```
print(Fruits)    # Output: Apple  
print(fruits)    # Output: Banana
```

Rule 5:

- A identifiers name **cannot be Python reserved Keywords.**

✓ Valid

```
total_sum = 100  
class_grade = "A"
```

✗ Invalid

```
if = 20  
class = "Math"
```

Both are **Python Reserved Keywords**

Python Reserved Keywords

The following identifiers are used as
reserved words

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Summary Of Rules

Rule	✗ What NOT to Do	✓ Valid Example
✓ Start with a letter or _	Don't start with a number (1var)	my_var, _count
✓ Use only letters, numbers, _	Don't use symbols like @, -, or space	hello123, user_1
✓ No spaces in variable names	Don't use space (total score)	total_score, first_name
✓ Case-sensitive	Don't assume Score and score are same	Score ≠ score
✓ Don't use reserved keywords	Don't name variables if, class, def	my_class, def_name

Common Mistakes

Reserved Keywords

- Python has some **words that are already used for its own rules**. These are called **reserved keywords**.
- Don't **use Python keywords** like **if, class, def, for, etc.**

✗ Invalid

```
if = 10
```

-----> Reserved Keyword

✓ Valid

```
condition = 10
```

Case Sensitivity

- Python is **case-sensitive**. That means **Age** and **age** are treated as different variables.
- Always **use the same lowercase format** to avoid mistakes.

✗ Invalid

```
Age = 20  
age = 25
```

✓ Valid

```
user_age = 25
```

Avoid Using Confusing Names

- Short names like **nm**, **dt**, or **x1** are **hard to understand**.
- Use **full words** that **explain** what the **variable is for**.

✗ Invalid

```
nm = "John"
```

✓ Valid

```
name = "John"
```

Avoid Overly Long Variable Names

- Very **long variable names** are **hard to read and manage**.
- Be **clear**, but **keep it short**.

✗ Invalid

```
number_of_students_enrolled_in_the_course = 50
```

✓ Valid

```
student_count = 50
```

Best Practices for Naming Variables

- When we write code, our **variable names should be clear and easy to understand**.
- **Good names** make the **code easier to read, fix,** and **work for team too**.

Here are some best practices we should follow

- Use Descriptive and Meaningful Names
- Use snake_case (Lowercase with Underscores)
- Be Consistent
- Avoid Using Single Letters
- Avoid Starting Variable Names with Numbers
- Use a Consistent Prefix or Suffix for Similar Variables

Use Descriptive and Meaningful Names

- ◆ Use names that tell you what the variable is for.

✗ Invalid

```
a = 25
```

- What does '**a**' **mean** ?

✓ Valid

```
age = 25
```

- **clearly indicate** that variable **hold age**

Use snake_case (Lowercase with Underscores)

- ◆ This makes your code easier to read and keeps it standard.

✗ Invalid

```
totalSales = 1000 #
```

- CamelCase is **less common** in **python**

✓ Valid

```
total_sales = 1000
```

- **snake_case** is **preferred style** in **python**

Be Consistent

- ◆ Choose one naming style and use it throughout your code.

✗ Invalid

```
total_sales = 1000  
TotalCost = 500 --> # Mixing styles
```

✓ Valid

```
total_sales = 1000  
total_cost = 500 # Both follow snake_case
```

Avoid Using Single Letters

- ◆ Short names don't explain what the variable is doing., instead use full words. that explain the purpose.

✗ Invalid

```
t = 75 # What does 't' mean?
```

✓ Valid

```
temperature = 75 # Clear and readable
```

Avoid Starting Variable Names with Numbers

- ◆ Python **does not** allow variable names to start with a number. start with letter or nuumbers

✗ Invalid

```
2nd_place = "Alice"
```

✓ Valid

```
second_place = "Alice"
```

Use a Consistent Prefix or Suffix for Similar Variables

- ◆ When you have related variables, use prefixes or suffixes to group them logically.

✗ Invalid

```
salesQ1 = 2000  
profit = 500
```

```
# Not clear if this is for Q1 too
```

✓ Valid

```
sales_q1 = 2000  
profit_q1 = 500
```

```
# Grouped by '_q1'
```

Multiple Assignments

- Multiple Assignments
- Re-declaring a variable
- Assigning multiple values to multiple variables
- Assigning the same value to multiple variables

Multiple Assignments

- In Python, we **use the = sign to assign values to variables.**

```
a = 100
b = 200

print(a) # Output: 100
print(b) # Output: 200
```



We can assign values in different ways

1. Re-declaring a variable
2. Assigning multiple values to multiple variables
3. Assigning the same value to multiple variables

Re-Declaring Variables

- We can change the value of a variable at any time. This is called re-declaration.
- Python lets you update a variable by just assigning a new value to it.

```
# Initial value
age = 25 -----> Original
print("Initial age:", age)

# Updating the value
age = 30 -----> Modified
print("Updated age:", age)
```

What's happening

Variable	Value Before	Value After
age	25	30

Point to remember

- The **variable name** stays the same
- The **value** changes
- The **old value** is removed from memory

Concept Practice Task

Re-declaring a Variable with a New Value

```
x = 10  
  
x = 20  
  
print(x)  
# output: 20
```

Re-declaring with a Different Type

```
a = 50  
  
a = "Python"  
  
print(a)  
# output: Python
```

Re-declaring a Variable Multiple Times

```
age = 25  
  
age = 30  
  
age = 35  
  
print(age)  
# output: 35
```

Re-declaring Variables at Different Points in the Program

```
name = "Alice"  
print(name) # Output: Alice  
  
name = "Bob"  
print(name) # Output: Bob
```

Assign Multiple Values to Multiple Variables

We can assign more than one value at once by using commas.

Basic Example

```
a, b = 100, 200
```

```
print(a) # 100
```

```
print(b) # 200
```

Here, **a** gets 100, and **b** gets 200.

Example with Different Data Type

```
a, b, c = 0.1, 100, 'string'
```

```
print(a) # 0.1
```

```
print(b) # 100
```

```
print(c) # string
```

Python allows mixing data types like float, int, and string.

! Important: Variable Count Must Match Value Count

✗ Too Many Values

```
a, b = 100, 200, 300
```

3 values

2 Variables

```
PS C:\Users\HP\Desktop\tt> & "C:/Program Files/python.exe" c:/Users/HP/Des
Traceback (most recent call last):
  File "c:\Users\HP\Desktop\tt\test.py", line 1, in <module>
    a, b = 100, 200, 300
ValueError: too many values to unpack (expected 2)
PS C:\Users\HP\Desktop\tt>
```

✗ Not Enough Values

```
a, b, c = 100, 200
```

2 values

3 Variables

```
PS C:\Users\HP\Desktop\tt> & "C:/Program Files/python.exe" c:/Users/HP/Deskt
Traceback (most recent call last):
  File "c:\Users\HP\Desktop\tt\test.py", line 1, in <module>
    a, b, c = 100, 200
ValueError: not enough values to unpack (expected 3, got 2)
PS C:\Users\HP\Desktop\tt>
```

● Tip: Always keep the number of variables and values equal.

Assign the Same Value to Multiple Variables

- We can assign one value to many variables using = repeatedly.

Basic Example

```
a = b = 100  
  
print(a) # 100  
print(b) # 100
```

- Both **a** and **b** get the **same value: 100**.

Assigning value to three or more variables.

```
a = b = c = 'string'  
  
print(a) # string  
print(b) # string  
print(c) # string
```

- **All three variables hold the value 'string'.**

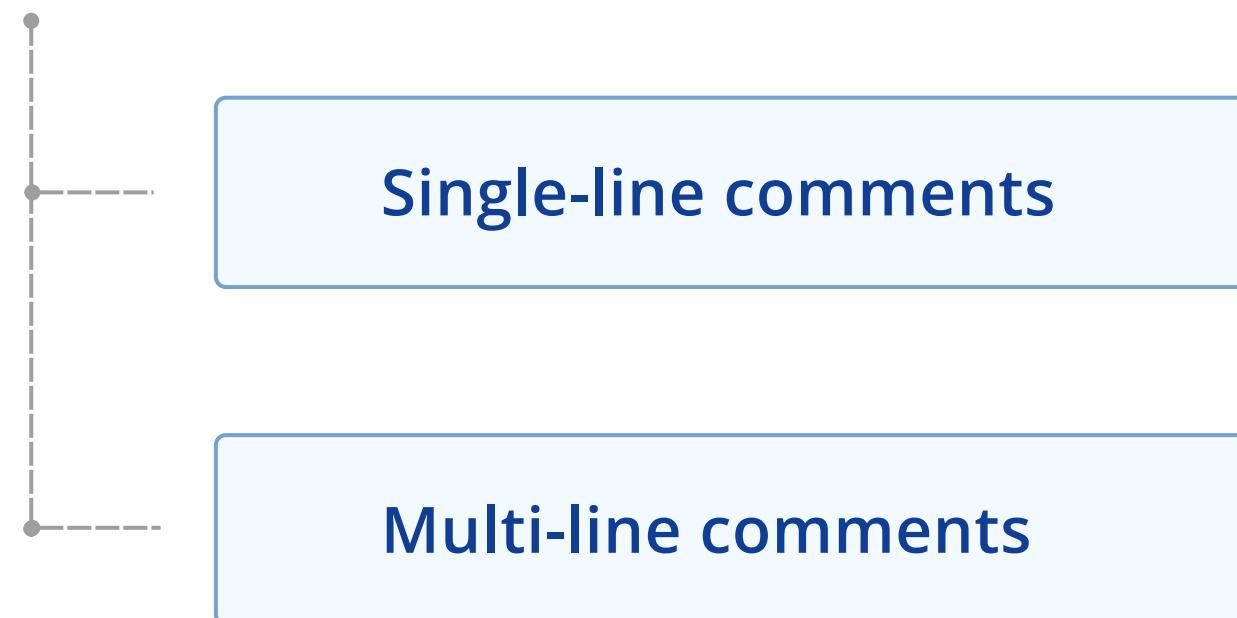
Comments in Python

- Comments in Python
- Single-line comments
- Multi-line comments
- Why Are Comments Required ?
- How to Comment and Uncomment a Line of Code in VS Code

Comments in Python

- Comments **are like notes in code.**
- They are there to **help us or others understand what our code is doing.**
- **Python ignores comments** when running the program.

Comments are of two types:



Single-line Comments

- A single-line comment is written on one line. It starts with the # symbol.

```
# This is a single-line comment.  
print("Hello, world!") # This line of code prints a message.
```

- Anything after **#** is treated as a comment.
- **Python skips Comments** when running the code.
- Use single-line comments to **explain short things like what a line of code does.**

Concept Practice Task

Simple Arithmetic Operations

```
# Define two numbers
a = 10
b = 5

# Perform addition
result = a + b

# Print the result
print(result) # This will output 15
```

Explaining Variable Assignment

```
# Assign the length and width of a rectangle
length = 10 # Length of the rectangle
width = 5   # Width of the rectangle

# Calculate the area
area = length * width

# Print the area
print(area) # This will print 50
```

Commenting Out Code

```
# This line of code is commented out and won't be executed
# print("This won't be printed")

# This line will be executed
print("This will be printed")
```

Multi-line Comments

- A **multi-line comment** is used when **you want to write a longer note over several lines**.
- It is **written between triple quotes** (""" """ or ''' ''').

```
"""  
This is a multi-line comment.  
It explains something in more detail.  
It can go over many lines.  
"""  
print("Hello, world!")
```

- Everything **inside the triple quotes is a comment**.
- **Python skips it during execution**.

Concept Practice Task

Comments for Multiple Variables

```
"""
In this example, we define variables to store information about a person.
We include details such as their name, age, and city of residence.
"""

name = "John"
age = 35
city = "New York"

"""
Now we will print this information to the console.
"""

print(name)
print(age)
print(city)

#Output:
John
35
New York
```

Multi-line String for a Comment Block

```
'''
This is a multi-line comment.
It can span across multiple lines.
This comment explains the purpose of the following code.
'''

# Defining two variables
x = 10
y = 5

# Adding the variables
result = x + y

# Printing the result
print(result) #Output: 15
```

Concept Practice Task

Explaining Multiple Sections of Code

```
"""
Step 1: Initialize variables for a basic arithmetic operation.
"""
# Define two numbers
a = 12
b = 6

"""
Step 2: Perform arithmetic operations.
"""
# Perform addition
addition = a + b

# Perform subtraction
subtraction = a - b

"""
Step 3: Display the results of the operations.
"""
# Print the results
print(addition)    # 18
print(subtraction) # 6
```

Why Are Comments Required?

- Comments play a very **important role in writing clean, understandable, and maintainable code.**
- Main Reasons Why Comments Are Needed:
 - **Explain what the code is doing**
 - **Make the code easier to read**
 - **Help others understand the code later**
 - **Support teamwork and collaboration**

Intent

- Sometimes **variable names don't fully explain** what they're used for.
- **A comment tells us why the code is written that way.**

✓ Good Comment

```
# Price of a single item in USD  
item_price = 50
```

✗ Bad Comment

```
# item price  
item_price = 50
```

Provide Context

- Some values or limits in the code need background info.
- A comment can tell why a certain number is used.

✓ Good Comment

```
# Maximum number of items a customer can purchase in one order  
max_items_allowed = 10
```

✗ Bad Comment

```
# 10 items  
max_items_allowed = 10
```

Clarify Expressions

- Sometimes, even basic operations involving variables can be unclear.
- Comments help clarify what certain expressions or calculations represent.

✓ Good Comment

```
# Total price after applying a 10% discount  
total_price = item_price * 0.9
```

✗ Bad Comment

```
# threshold  
discount_threshold = 100
```

Maintain and Update

- **Well-commented code** makes it easier to **maintain** and **update**.
- **Developers** returning to code **after a long time may forget why certain variables were named or used** in a particular way. **Comments act as reminders**

✓ Good Comment

```
# Discount threshold: prices above this value get a discount  
discount_threshold = 100
```

✗ Bad Comment

```
# threshold  
discount_threshold = 100
```

Better Collaboration

- When working in teams, not everyone will know code logic.
- A **short comment helps teammates understand your code faster**

✓ Good Comment

```
# The default number of items displayed on a product page  
items_per_page = 20
```

✗ Bad Comment

```
# display items  
items_per_page = 20
```

How to Comment and Uncomment Code in VS Code

Single-Line Comments

To comment one line in Python,
add a # at the beginning of that line.

```
# This is a comment  
tax_rate = 0.07
```

Comment Multiple Lines (Block Commenting)

- **Select the lines of code** you want to comment
- Open VS Code Press:
 - **Ctrl + /** on Windows/Linux
 - **Cmd + /** on Mac

```
# item_price = 100  
# total_price = item_price * 0.9  
# tax_amount = total_price * 0.07
```

To uncomment:

- Select the commented lines
- Press the same shortcut again:
- **VS Code will remove the # symbols.**

Common Errors with Variables

- Syntax Errors
- Name Errors
- Type Errors

Common Errors with Variables

Syntax Errors

- A **syntax error happens** when the **code is written in a way that breaks Python's rules**.
- Common causes
 - Using **invalid characters in variable names**
 - **Starting** a variable name **with a number**
 - **Using spaces** in variable names

✗ Cause

```
my-variable = 10  —→ # Error: "-" is not allowed  
1name = "John"   —→ # Error: Variable name can't start with a number
```

✓ Fix

```
my_variable = 10  —→ # Use underscores instead of dashes  
name1 = "John"   —→ # Start with a letter
```

Name Errors

- A **NameError** happens when **Python can't find the variable we are trying to use.**
- Common causes:
 - **Using a variable before defining it**
 - **Spelling mistakes in variable names**

✗ Cause

```
print(age)
```

Here age is not define

```
PS C:\Users\HP\Desktop\tt> & "C:/Program Files/python.exe" c:/Users
Traceback (most recent call last):
  File "c:\Users\HP\Desktop\tt\test.py", line 1, in <module>
    print(age) # Error: 'age' is not defined yet
NameError: name 'age' is not defined
PS C:\Users\HP\Desktop\tt>
```

✓ Fix

```
age = 25
print(age)
```

```
PS C:\Users\HP\Desktop\tt> &
25
PS C:\Users\HP\Desktop\tt>
```

Type Errors

- A `TypeError` happens when you **try to mix values of different data types** in a way that Python doesn't allow.
- **Common causes:**
 - **Adding a string** and an **integer** together
 - Using variables in operations that **don't match their type**

✗ Cause

```
age = 25
greeting = "Hello"
result = age + greeting
```

```
PS C:\Users\HP\Desktop\tt> & "C:/Program Files/python.exe" c:/Users,
Traceback (most recent call last):
  File "c:\Users\HP\Desktop\tt\test.py", line 3, in <module>
    result = age + greeting
TypeError: unsupported operand type(s) for +: 'int' and 'str'
PS C:\Users\HP\Desktop\tt>
```

✓ Fix 1 – Convert types

```
result = str(age) + greeting # Output: '25Hello'
```

✓ Fix 2 – Use a comma in print():

```
print(age, greeting) # Output: 25 Hello
```

Thank You

